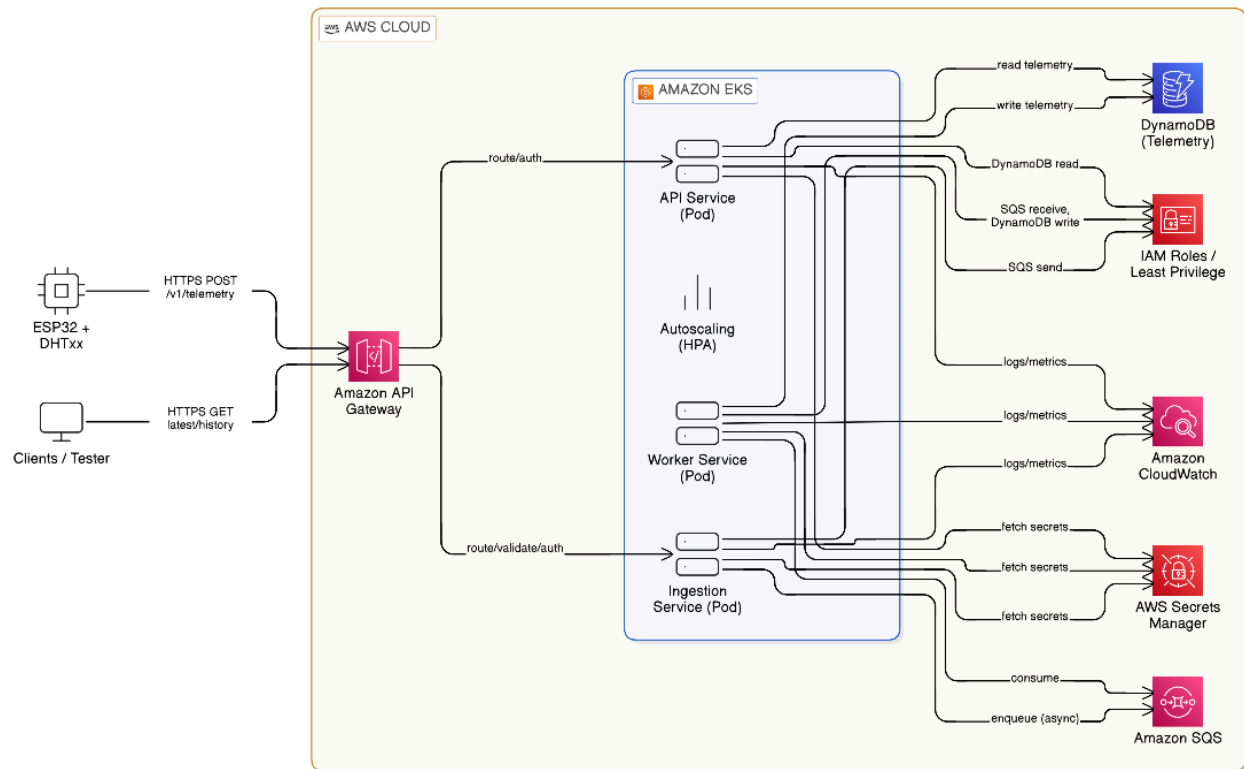


Project Architecture Package
Cloud-Based IoT Environmental Telemetry Platform
Seid Cubro - Charles Shoppel

1. Architecture Diagram:



2. Components and Responsibilities:

Amazon API Gateway (HTTP API)

- TLS termination and routing
- Authentication enforcement (API key or JWT)

Ingestion Service (Amazon EKS)

- Accept telemetry from ESP32
- Validate schema and required fields
- Publish telemetry event to SQS
- Respond quickly

Amazon SQS

- Buffer telemetry events
- Enable async processing and reliability
- Support failure scenarios

Worker Service (Amazon EKS)

- Consume messages from SQS
- Normalize, validate values
- Write to DynamoDB
- Retry on failure, log errors, emit metrics

API Service (Amazon EKS)

- Read telemetry from DynamoDB
- Provide REST endpoints for latest and historical data
- Enforce authorization if needed

3. Data Flow:

Flow A: Telemetry ingest (ESP32 to AWS)

1. ESP32 reads DHT sensor
2. ESP32 sends POST /v1/telemetry to API Gateway
3. API Gateway routes to Ingestion Service (Amazon EKS)
4. Ingestion validates and writes message to SQS
5. Worker consumes from SQS and writes to DynamoDB

6. Logs and metrics go to CloudWatch

Flow B: Read latest

1. Client calls GET /v1/devices/{deviceId}/telemetry/latest
2. API Gateway routes to API Service (Amazon EKS)
3. API Service queries DynamoDB
4. Returns JSON

Flow C: Failure demo

- DynamoDB throttling or worker failure → queue depth grows
- CloudWatch alarm triggers on ApproximateNumberOfMessagesVisible
- Recovery: fix worker / scale it → drains queue

4. Key Design Decisions and Trade Offs:

- DynamoDB vs. RDS
 - Chosen: DynamoDB
 - Why: serverless scaling, no schema migrations, fits time-series-per-device pattern well
 - Trade-off: complex ad-hoc analytics queries are harder than SQL, you design access patterns up front
- API Gateway and EKS
 - Why: clean separation (gateway policies/auth) and Kubernetes required by milestones
 - Trade-off: slightly more setup than using ALB-only ingress
- SQS async processing
 - Why: resilience, spike absorption, clean failure demos
 - Trade-off: eventual consistency

5. API Sketch:

POST /v1/telemetry (device ingest; returns 202)

GET /v1/devices/{deviceId}/telemetry/latest

GET /v1/devices/{deviceId}/telemetry?start=...&end=...

GET /health

6. DynamoDB Data Model and Access Patterns:

The system uses Amazon DynamoDB to store time-series telemetry data per device. The table is designed around predictable access patterns rather than ad-hoc queries.

Table Design:

- Partition Key: device_id
- Sort Key: timestamp (ISO-8601 string)

Access Patterns:

- Latest telemetry:
Query by device_id with ScanIndexForward=false and Limit=1
- Historical range query:
Query by device_id with timestamp BETWEEN start AND end
- Device isolation:
Each device's data is fully partitioned by device_id, enabling horizontal scalability

Data Lifecycle:

- TTL:
A TTL attribute may be used to automatically expire older telemetry records if retention limits are required.

This design supports high write throughput, efficient per-device reads, and aligns with DynamoDB best practices for time-series workloads.

7. Authentication and Secrets Management:

For this project, API key–based authentication is selected for device ingestion.

Authentication Choice

- Selected: API Key
- Why:
 - Simple for ESP32-based devices
 - Supported natively by Amazon API Gateway
 - Sufficient for course scope and threat model

Validation

- API Gateway validates API keys on incoming requests before routing to backend services.

Secrets Management

- API keys and sensitive configuration values are stored in AWS Secrets Manager
- Kubernetes workloads access secrets using IAM roles for service accounts (IRSA)

Rotation Strategy

- Keys can be rotated manually by creating a new key in API Gateway and updating Secrets Manager without redeploying services.

8. Observability Metrics and Alerting:

The platform uses Amazon CloudWatch for centralized observability across services.

Key Metrics

1. API 5xx Error Rate

- Indicates service instability or downstream failures
- 2. Request Latency (p95)
 - Measures API responsiveness
- 3. SQS Queue Depth (ApproximateNumberOfMessagesVisible)
 - Detects ingestion backlogs and worker failures
- 4. Worker Error Rate
 - Tracks failed message processing attempts
- 5. Pod CPU Utilization
 - Drives Horizontal Pod Autoscaling (HPA)

Sample Alert Thresholds

- Queue depth > 100 messages for 5 minutes
- API 5xx rate > 2%
- Worker error rate > 1%
- p95 latency > 1 second

These thresholds serve as initial placeholders and can be tuned during load testing in M8.

9. Network and Security Boundaries:

The system is deployed within a VPC with clear separation between public and private components.

Traffic Flow

- TLS termination: Amazon API Gateway
- Public access: Only API Gateway is publicly accessible

- Private services: EKS worker nodes and pods run in private subnets

Ingress Handling

- API Gateway routes traffic to EKS services using private networking
- Kubernetes Ingress handles internal service routing

Security Controls

- Security groups restrict inbound traffic to required ports only
- DynamoDB is accessed via IAM roles (no static credentials)
- All inter-service communication occurs over HTTPS

This design minimizes the attack surface and enforces least-privilege access throughout the system.